Canaries and Stack Guards

Agaton Pirander & Oliver Marcusson

October 2026

1 Introduction

Buffer overflows are one of the oldest vulnerabilities and one still used today. Due to its age and the plethora of security measures built to defend against it, it is not as prevalent as it once was. Some of these security measures are ones implemented in this project.

This project consists of two security implementations that aim to protect the stack of S3K, built on the RISC-V infrastructure, against buffer overflows. Canaries and Stack-Guards are well established concepts and are security measures that complement each other and enhance the protection of the control flow for a running process. Canaries and stack-guards in this implementation protect against buffer overflow attacks. Canaries protects against overwrites of the return address through a value placed on the stack between a buffer and the return address while the guard pages make memory segments non-writable and non-readable enhancing protection against more sophisticated buffer overflow attacks.

2 Stack Canaries

Stack canaries are a randomized value made to be infeasible to guess that is placed between the stack buffer and the control data for the function frame. Through a control function that reads the canary value it can make decisions on if the stack control data has been overwritten from a buffer or not. Possibly stopping and causing an error when the canary value differs after a buffer write. The security function depends on the attacker not being able to read or figure out the value of the canary as they then can write the same value when overwriting to bypass the protection. In GCC when compiling binaries one can choose to add stack canaries using compile flags which adds additional functions between calls that test the controlflow.

3 Stack guards

Stack guards or guard pages are spaces in memory removed of permissions causing an error if the program tries to read, write, or execute. Depending

on the type of stack guard in place, it might only want to limit execution placing a region under no execute or no write but it is commonly used to protect spaces that the current control flow should have no access to. Guard pages are commonly placed at the end of the stack, to stop a stack that grows into other memory. In this project, we instead place guards inside every function frame. When the program tries to read to a place forbidden by the kernel, it causes an error stopping the program and the improper control flow.

4 Technical Analysis

The project encompasses both the canary function and guard pages. The guard pages are placed inside of stack frames to protect both the canary, the return address, and the frame pointer. This is done to layer the protection against more sophisticated buffer overflow attacks. The guard pages protect values and memory spaces inside of each frame, the lack of permissions in certain memory spaces hinders all operations accessing them as the process logically cannot access them when interacting with the kernel because of the lack of PMP allocation.

4.1 Canary

The kernel and all applications are compiled with GCC's -fstack-protector-strong. GCC then places a canary between the local buffers and the saved return address of every function that has an array on the stack, and checks it before the function returns. We supply our own canary value and failure function instead of relying on a standard library, since S3K has none. The value is generated at boot from the timer, an address and the hart ID, mixed with a hash function, and its lowest byte is set to zero so a string copy that stops at a NUL byte cannot write it back. If the check fails, the program prints "*** stack smashing detected ***" and stops. The canary only detects an overflow when the function returns, after the return address has already been overwritten, which is why the stack guards are added on top of it.

4.2 Stack guard

RISC-V restricts memory access with Physical Memory Protection (PMP), and S3K gives each process 8 PMP entries. Two are used for the program and the serial port, which leaves six for the stack. That is far too few for one guard page per frame, so we instead took a virtualized approach: the stack has no permanent mapping at all, so every first access to it faults. The monitor keeps a table of everything that must never be touched, namely the canary, the saved return address and the frame pointer of every active frame. On each fault, the monitor checks the address. If it is a normal stack access, the monitor maps a small window of the stack around it that does not cover any guard, and lets the protected application continue. If the address is a guard, an instruction

fetch from the stack (shellcode), or outside the stack, the monitor treats it as an attack and leaves the application suspended. An overflow therefore stops at the first byte that leaves the buffer, before anything is overwritten, and the decision is made by a process the attacker cannot reach.

5 Vulnerabilities

With only a canary implemented as a security feature there are a number of ways to bypass it that stack guards protect against. This project targets these types of stack-based memory corruption in the vuln app.

- Linear stack-buffer overflow. The most common type of buffer overflow where too much data is written to a buffer that overflows and overwrites the returnaddress of the frame, or other data on the stack. This is stopped by the normal canary implementation provided that the canary value is not leaked through other exploits.

- Non-linear stack-buffer overflow or indexed buffer overflow. This gives the attacker the ability to make non-contiguous writes in memory effectively bypassing the canary. The guard pages protect the return addresses from these types of exploits hindering execution, however other data may still be edited.

- Canary disclosure. With only the canary implementation the canary value can be read through memory leaks or exploits. Guard pages hinder this because the memory guard is without any memory privileges during execution, hindering any access to the canary value during process execution.

- Overflows through a pointer. A pointer to a buffer in an earlier function's frame can be used to overflow that frame instead of the current one. The guard arrays hold every active frame, not only the current one, so the earlier frame's canary and return address are still protected.

6 Design

The project is divided into 3 applications run through the S3K kernel. The app0 setup process, the monitor process, and the vuln process. The app0 sets up each of the following processes capabilities and starts them.

Monitor is responsible for keeping track of the stack guards in memory, handling the memory faults that the vulnerable process gets with new memory access and setting up the memory PMP windows when deciding if the memory access is allowed or not. This is the privileged process.

Vuln is the vulnerable process that is susceptible to buffer overflow. It has extra GCC compiler flags enabled to give power over the control flow; -finstrument-functions and -fstack-protector-strong. Using hooks in the control flow the process sends stack frame information using IPC to the monitoring process, this information is currently partially supplied by the user post-compilation.

7 Limitations

The implementation of stack guards causes large amounts of overhead because each function call and many memory reads need to go through extra security steps. With a larger stack filled with many frames the low amount of PMP slots available will cause each new function to remove earlier PMP memory segments, this in turn results in reapplying the PMP when the control flow returns back to the earlier frames. The result is that this implementation is meant for smaller proof of concept programs. There is also a hardcoded amount of possible concurrent stackframes which is currently 64.

8 Personal Contribution

8.1 Agaton Pirander

8.1.1 Personal Canary Implementation

In this project the canary implementation that Marcusson wrote has been the origin. My implementation is found in the project repository but is not part of our final stack guard implementation. My canary is made using the GCC implementation of canaries except the functions are changed from the default configuration. Using the stack_chk_guard function the canary value is static in the implementation as it is a proof of concept and does not need to be infeasible to guess. The consequence of the canary being changed during runtime for my implementation was to suspend the time capability of the running vulnerable app. This causes the process to be suspended indefinitely or until the program starts again. The design used in the canary is similar to the memory guards implemented where IPC is used between the vulnerable app and the monitoring app. The monitoring app has privileges over the vulnerable app and can make decisions based on information received.

8.1.2 Capturing the Frame

Because of the limitation of memory segments it is not possible to have one or two stack guards per stack frame. Instead the project allocates allowed memory segments that does not overlap with the addresses of the canary or return address. Information on the stackframes must therefore be stored to make decisions on where memory can be given read/write privileges. The implementation uses -finstrument-functions that hook at the beginning and end of each affected function to capture the frame_high, function address, and the locations of stack guards. Due to time constraints, the frame sizes used together with the stack pointer to find the stack_high and the location of the canary are found statically by the programmer after compilation and placed in an array. This enables recompilation with automated stack offsets. In the RISC-V architecture the arguments of a function are passed using the a registers. To save the stack pointer before it is edited by the instrumentation functions it had to be captured using

an assembly function that saves the stack pointer to the third argument passed as the other two already are populated by function address and caller address.

As part of the implementation of the guard frames, certain information is necessary. Because the implementation is restricted by only 6 allocated PMP slots the project uses information stored in arrays to allow PMP slot allocation across memory where canaries and return address is not placed. That they will never be given rw permissions. The PMP allocation is made for memory that is allowed to be accessed, not for memory not allowed to be accessed. This makes it so that the forbidden memory areas can be defined by the arrays in place and not allocated PMP. Never allocating memory segments that overlap with the values in the arrays present.

8.1.3 Creating Frame and Guard Arrays

The frame information is captured by the vuln app and sent using IPC to the monitor application that populates arrays with information of the frame and guards. The vuln app sends the stack_high value by adding the stack offset to the stack pointer given from the finstrument instructions. It also sends the canary address present in the frame. Two arrays are created with the information. One for each frame, and one for each of the guards present in that frame. To populate the arrays the functions guard_push and guard_pop are used. The two arrays use the same index counter which makes it easy to pop the correct frame and know which frame is accessed. Removing an item using pop reduces the index counter which makes the popped frame's entries free to be overwritten by the next push, so they never need to be cleared.

8.1.4 PMP allocation

Once the frame and guard arrays are in place it is possible to allocate memory segments that the program is allowed to access. Once the process tries to access a memory location it will cause an error that is caught by the fault handler. This fault handler can then call the function window_map to control the memory segment and load PMP to the proposed memory address. The function tries to allocate the largest possible memory segment on the stack without hitting any forbidden addresses. This is to keep the need for additional PMP allocation per frame to the minimal. This implementation only protects against overflows against the canary and the return address, other parts of the frame can still be overwritten.

8.1.5 AI Use

I have used AI throughout this project to gather knowledge about S3K as well as compilation troubleshooting and coding questions. I have asked about C functionality and the builtin functions in S3K, some examples being IPC, PMP and writing assembly functions in C. Many times it has been my lack of coding experience in C that has been the issue resulting in both googling and AI searching.

I have also used the tutorial examples to copy the S3K functions when necessary to know which parameters to populate and the ordering of certain capability allocations.

8.2 Oliver Marcusson

8.2.1 Canaries and the exploit

I added stack canary support to S3K and it's applications, making use of the previously mentioned GCC compiler flag -fstack-protector-strong. This also includes the generation of the canary value at boot before any protected function runs, as well as separate failure handlers for the kernel and the applications. They can not be the same as that would cross the border between machine and user-mode. The initialization function itself however has to be compiled without a canary, because it modifies the value it would be checked against. I also created the vulnerable application and wrote the exploit, which is a simple partial overwrite of the return address, resulting in a ret2win.

8.2.2 Suspending instead of replying

Before building the guards, I wrote a small experiment to check wether S3K allows the monitor to change an applications memory capabilities while it waits for an IPC reply. The experiment showed that it does not. The kernel only allows it while the process is suspended. It however showed that the monitor can suspend the application, change its capabilities, and then resume it. In the end, the resume of the application became the communication protocol between the monitor and the application. A successful resume means the memory operation was allowed, and a suspension means that the operation was illegal.

8.2.3 Fault handling

I set up the memory layout where the stack has no permanent mapping, with the trap handler and its own stack placed in memory that is always mapped. Therefore, handling a fault can never result in a fault itself. The vulnerable application's trap handler forwards every memory fault to the monitor. In the monitor, I wrote the main loop and the fault classification, which decides if a fault is a normal stack access, a write to a guard, an access outside the stack, code executed from the stack, or a repeated fault, and logs the reason when it suspends the monitored application. For normal access, the monitor then uses the window allocation function written by Agaton to map the stack.

8.2.4 AI Use

I have used AI as a tool throughout the project to educate myself on what S3K is and how it operates under the hood. I have used it for brainstorming and helping with designing the project architecture, as well as the design of the utility library included in the project. I have also used AI to help with

proofreading and for giving feedback on this report. With the help of AI, I have multiplied my work efficiency and software development capabilities within this project.